

Module «Data Science»

Week 06: Supervised Learning



Content of this lecture

01

Foundations

What ML is · paradigms · map of methods · history

02

Model training and evaluation

Loss · train/test · cross-validation · bias–variance

03

Regression models

Linear regression · regression trees · random forest · boosting

04

Classification models

Logistic regression · classification trees · random forest & boosting · k-NN · Naive Bayes · SVM · evaluation

05

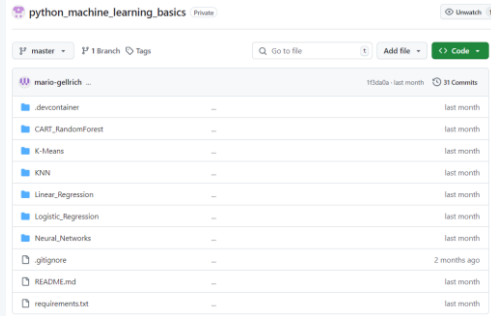
Unsupervised learning

k-means · hierarchical clustering · PCA

Hands-on

Every concept has a Python notebook in the course repository (scikit-learn).

Toolkit for the exercises



Course repository

[github.com/mario-gelrich-zhaw/
python_machine_learning_basics](https://github.com/mario-gelrich-zhaw/python_machine_learning_basics)

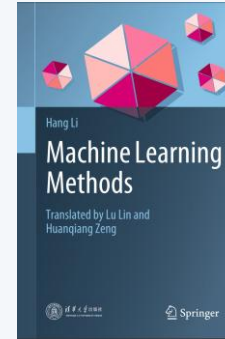
Fork it · open a Codespace



scikit-learn

scikit-learn.org

Consistent API: fit; supervised
models add predict/score, PCA
adds transform



Reading

[Li, H. \(2024\). Machine Learning
Methods. Springer.](#)

PDF via SpringerLink (ZHAW
network)

Foundations

Introduction

History

What is machine learning?

“A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P , if its performance at tasks in T , as measured by P , improves with experience E .”

— T. Mitchell (1997)

In data-science terms

- **Data:** n examples (rows), each with features x (e.g. area, rooms) and a known answer y (e.g. rent)
- **Goal:** learn a rule $\hat{y} = f(x)$ that also gives good answers for NEW examples

Typical applications

Price prediction

real estate, stocks

Predictive maintenance

will the machine fail?

Fraud & spam

transaction fraud
yes/no

Recommendation

bought $X \rightarrow$ also Y

Segmentation

customer groups

Perception

speech, images,
driving

Three learning paradigms

1 Supervised labelled data

pairs $(x, y) \rightarrow$ rule $\hat{y} = f(x)$

Learn a rule that predicts the known answer y from the inputs x .

Regression models $\rightarrow$ Part 3
Classification models $\rightarrow$ Part 4

2 Unsupervised unlabelled data

only $x \rightarrow$ groups / patterns

No answers given: find groups or patterns in the inputs x alone.

Clustering, PCA $\rightarrow$ Part 5

3 Reinforcement agent & environment

actions $\rightarrow$ rewards $\rightarrow$
strategy

An agent tries actions and learns a strategy that collects as much reward as possible.

Games, robotics, LLM fine-tuning
(not covered here)

Map of this lecture: three families of methods

	Regression models Part 3	Classification models Part 4	Unsupervised learning Part 5
Target y	a number (CHF, kg, minutes)	a category (yes/no, A/B/C)	none — only features x
Question	How much?	Which class?	Which groups / patterns?
Methods here	linear regression · regression tree · random forest · boosting	logistic regression · classification tree · random forest · boosting · k- NN · Naive Bayes · SVM	k-means · hierarchical clustering · PCA
Evaluation	RMSE, R^2 on test data	accuracy, precision, recall, ROC/AUC	no target used for fitting: elbow plot, stability, interpretation
Example	rent of a flat	survived the Titanic?	customer segments

Supervised learning: regression vs classification

y (label)	colour	height [cm]	weight [kg]
cat	yellow	42	2.5
dog	black	85	55.0
dog	yellow	35	4.5
cat	black	40	2.3

Labelled data: each row has features x (colour, height, weight) and a known target y .

KEY IDEA Same general workflow, different objectives: squared error for regression; log-loss, Gini/entropy or hinge loss for classification.

Regression

$$y \in \mathbb{R}$$

y can be any number

rent in CHF/m² · fuel consumption l/100 km · demand

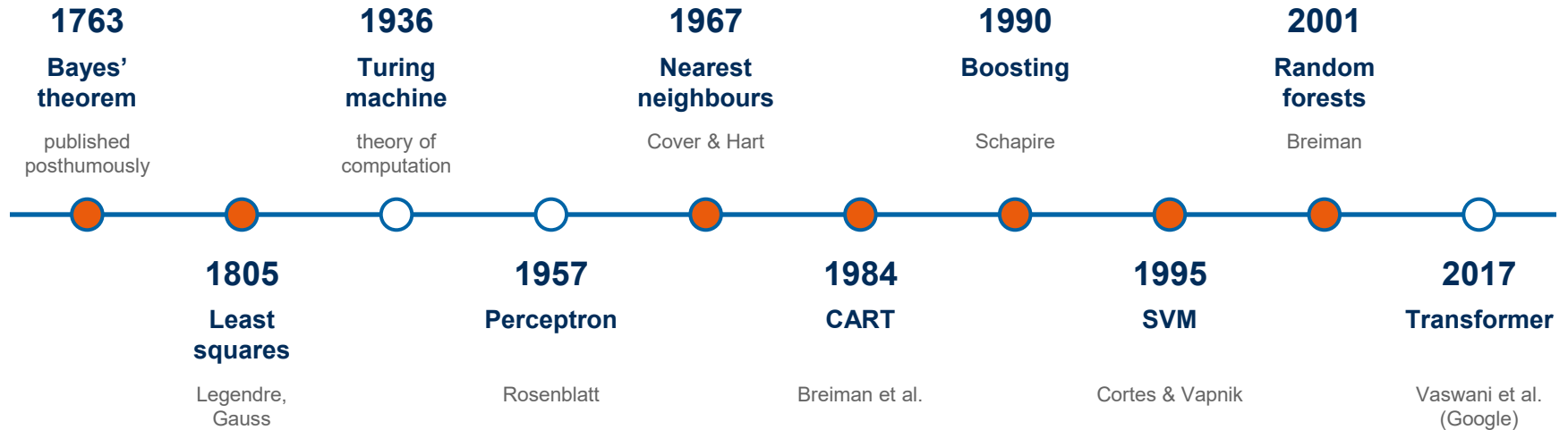
Classification

$$y \in \{1, \dots, K\}$$

y is one of K categories

survived yes/no · credit risk low/high · iris species

From Bayes to Transformers: a short history



 covered in this lecture

Model training and evaluation

Loss functions

Validation & overfitting

Loss functions: learning = minimising a loss

Training = make the average loss small

$$\text{training error} = \frac{1}{n} \sum_{i=1}^n L(y_i, \hat{y}_i)$$

In words: add up the loss of every training example and divide by n . Training picks the model with the smallest value.

Typical loss functions

$L(y, \hat{y}) = (y - \hat{y})^2$ squared loss: error 2 $\rightarrow$ 4, error 10 $\rightarrow$ 100 (big errors weigh heavily)

$L(y, \hat{y}) = \mathbf{1}\{y \neq \hat{y}\}$ 0-1 loss: 1 for a wrong class, 0 for a correct one

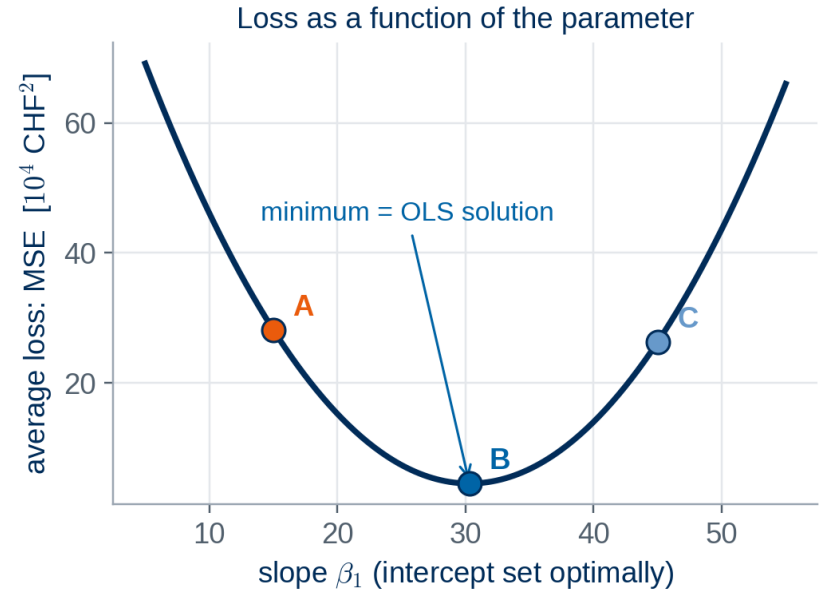
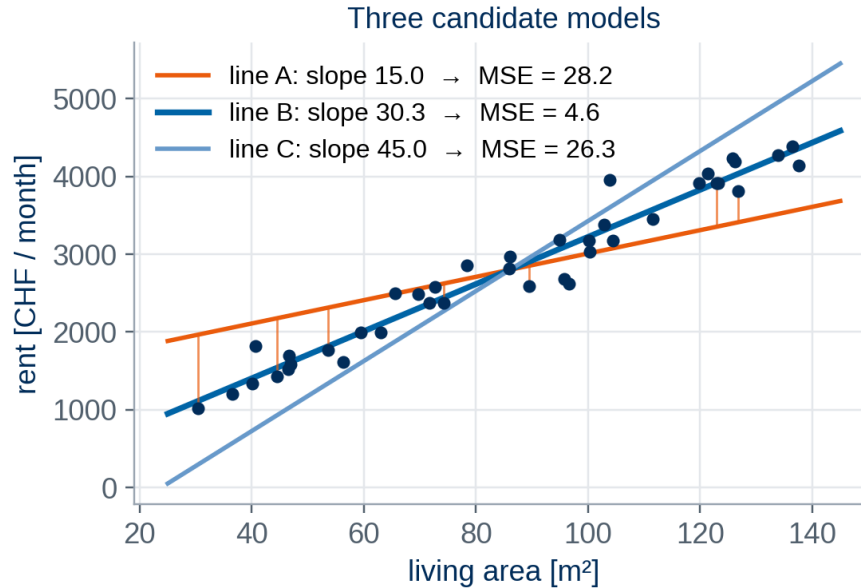
What we actually care about

**error on NEW, unseen data
(generalisation error)**

It cannot be computed directly — we estimate it on data that were not used for fitting (test set, cross-validation).

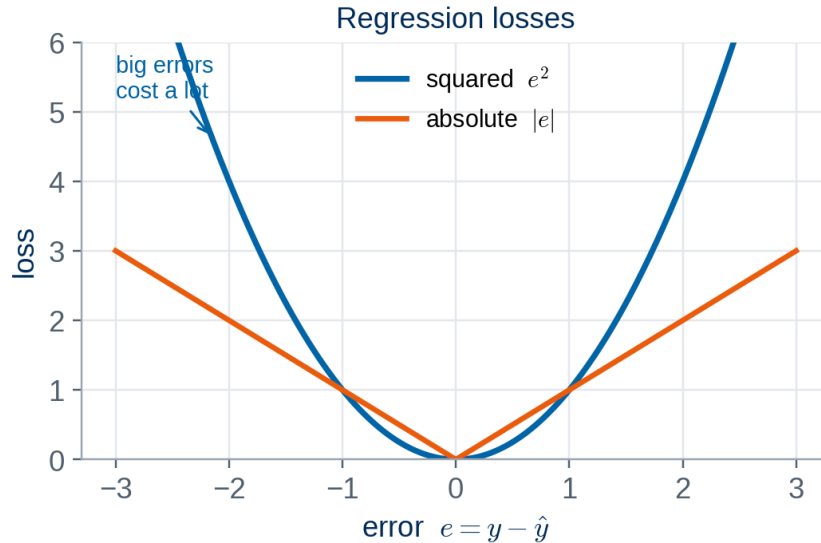
KEY IDEA Training error is typically optimistically biased: the same data were used to fit the model.

Loss functions: a graphical example

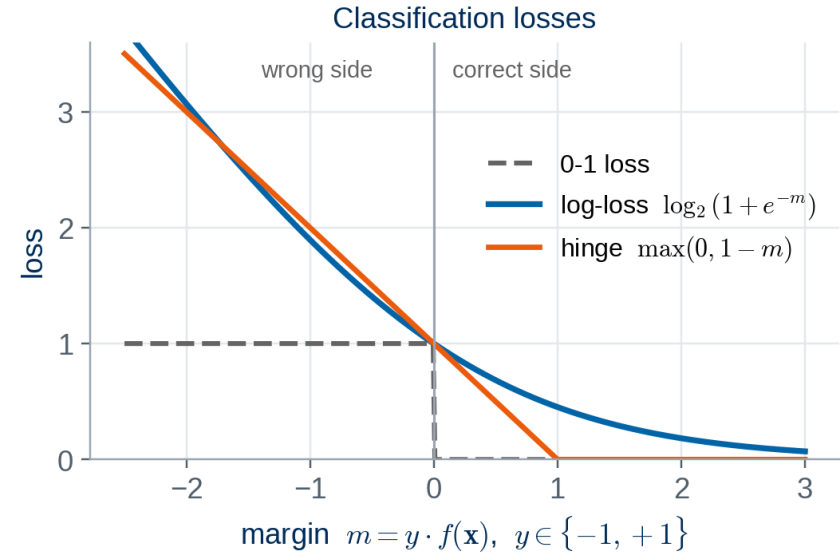


In words: every candidate model has an average loss. Training = searching for the parameter value at the bottom of the loss curve — here line B (the OLS fit) with the smallest MSE.

Loss functions: different losses, different penalties



In words: squared loss punishes large errors heavily (outlier-sensitive); absolute loss grows linearly (more robust).



Classification: $m > 0$ = correct side. Log-loss and hinge are convex stand-ins for the 0-1 loss; log-loss is smooth, hinge has a kink at $m = 1$. Both also penalise “barely correct” predictions.

Loss, objective and evaluation metric

Loss

error of ONE prediction

$(y - \hat{y})^2$, log-loss, hinge, 0-1

Training objective

what the algorithm optimises

average loss (+ penalty) or a split criterion

Evaluation metric

how we judge the model on new data

RMSE, R^2 , accuracy, F1, AUC

model	training objective (what is minimised)	typical evaluation metric
Linear regression (OLS)	sum of squared errors (RSS)	RMSE, R^2 on test data
Logistic regression	log-loss = negative log-likelihood	accuracy, F1, AUC
Decision tree (CART)	greedy: largest decrease in SSE / Gini / entropy per split	RMSE · accuracy, F1, AUC
SVM	hinge loss + l_2 penalty	accuracy, AUC
Gradient boosting	any differentiable loss (squared, log-loss, ...)	depends on the task

KEY IDEA The objective we optimise and the metric we report need not be the same.

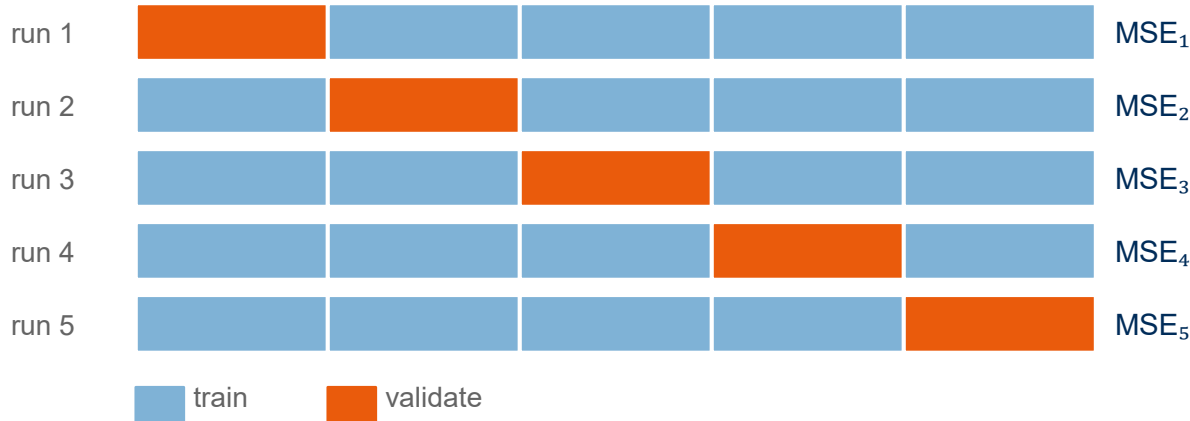
Train / test split and k-fold cross-validation

Hold-out split

training set (80 %) → fit the model, tune hyperparameters

test set (20 %)

k-fold cross-validation on the training set (k = 5)

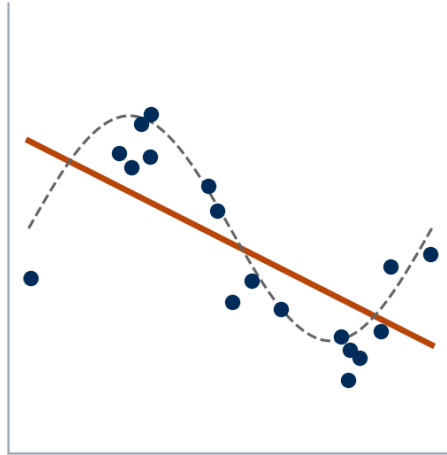


$$CV_{(k)} = \frac{1}{k} \sum_{j=1}^k MSE_j$$

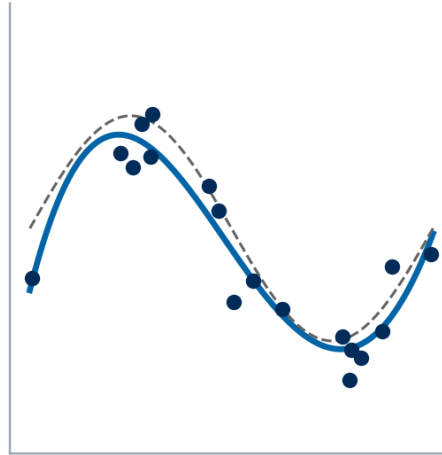
In words: the CV error is the average validation error across the k folds.

Underfitting and overfitting

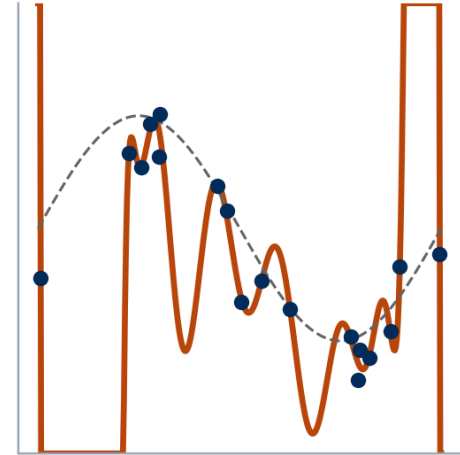
Underfitting (degree 1)



Good fit (degree 4)



Overfitting (degree 15)



Underfitting: Model too simple: high error on training and test data.

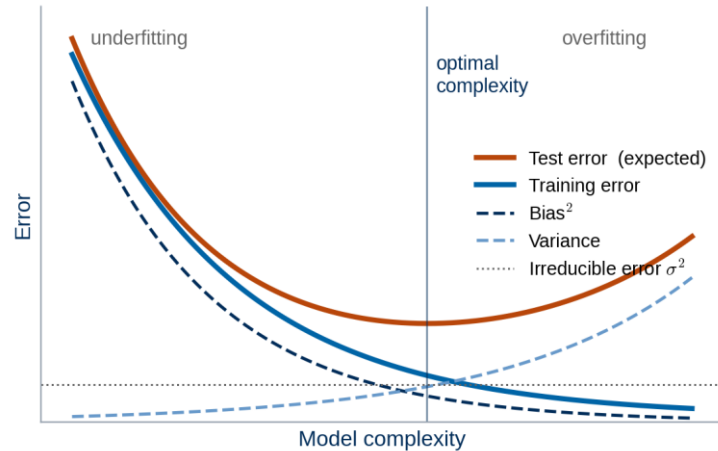
Good fit: Captures the signal, ignores the noise.

Overfitting: Learns the noise: low training error, high test error.

The bias–variance trade-off

$$\text{expected test error} = \text{bias}^2 + \text{variance} + \text{noise}$$

In words: error from too simple assumptions + error from instability across training samples + noise no model can remove (for squared-error loss).



- **Bias:** model too simple → systematically wrong
- **Variance:** model too flexible → changes a lot with another training sample
- **Goal:** minimise the sum — not either term alone

Example: predicting every rent by the overall mean → high bias; a degree-15 polynomial through 18 flats → high variance.

Regression models

Linear regression

Regression trees

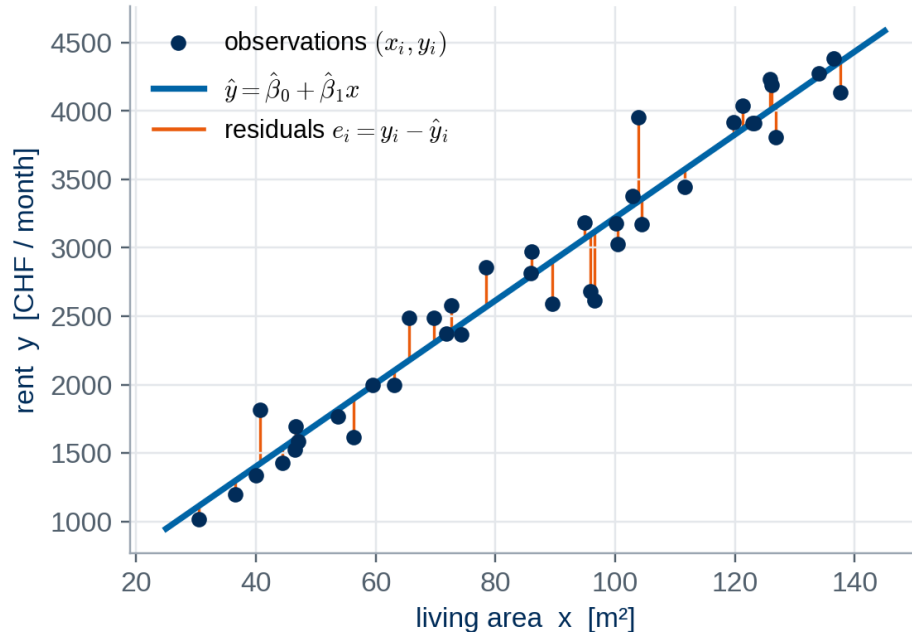
Ensembles: bagging

Random forest

Gradient boosting

Model comparison

Linear regression: the model



Simple linear regression

$$y_i = \beta_0 + \beta_1 x_i + \varepsilon_i$$

β_0 intercept β_1 slope ε error term

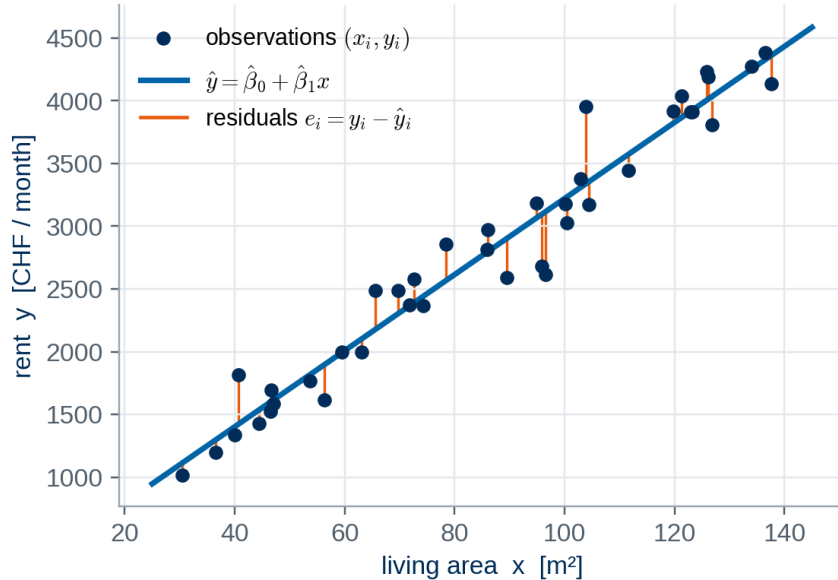
Multiple regression (several features)

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p + \varepsilon$$

In words: y is a weighted sum of the features plus a random error ε ; each β : change in the average y per unit of its feature, holding the other features constant.

KEY IDEA “Linear” means linear in the parameters β — x^2 or $\log x$ are allowed.

Linear regression: ordinary least squares (OLS)



1 Objective: smallest sum of squared residuals

$$RSS = \sum_{i=1}^n (y_i - (\beta_0 + \beta_1 x_i))^2$$

In words: square every orange residual and add them up; OLS chooses β_0, β_1 that make this sum as small as possible.

2 Solution (one predictor)

$$\hat{\beta}_1 = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}$$

$$\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$$

In words: slope = how strongly x and y move together, relative to the spread of x ; the line always passes through the point of means $(\bar{x}, \bar{y})$.

Linear regression: worked example (rent)

$$\widehat{\text{rent}} = 166.79 + 30.84 \cdot \text{area}$$

$$\widehat{\text{rent}}(95 \text{ m}^2) = 166.79 + 30.84 \cdot 95 = \mathbf{3096.59 \text{ CHF}}$$

Interpretation

- $\beta_1 = \mathbf{30.84}$: $+1 \text{ m}^2 \rightarrow +30.84 \text{ CHF}$ per month (on average)
- $\beta_0 = \mathbf{166.79}$: value at $x = 0$ — outside the data, no substantive meaning
- **Extrapolation** beyond the observed range of x is risky

3'097

CHF / month

predicted rent for a 95 m² flat

Linear regression: goodness of fit (R^2 , RMSE)

$$\underbrace{\sum (y_i - \bar{y})^2}_{\text{SST}} = \underbrace{\sum (\hat{y}_i - \bar{y})^2}_{\text{SSR}} + \underbrace{\sum (y_i - \hat{y}_i)^2}_{\text{SSE}}$$

In words: (OLS with an intercept) total variation of y = part explained by the model + unexplained rest

R^2 — share explained

$$R^2 = 1 - \frac{\text{SSE}}{\text{SST}}$$

0 = model explains nothing, 1 = perfect fit

Example: SSE = 20, SST = 100

$$R^2 = 1 - \frac{20}{100} = 0.8$$

RMSE — typical prediction error

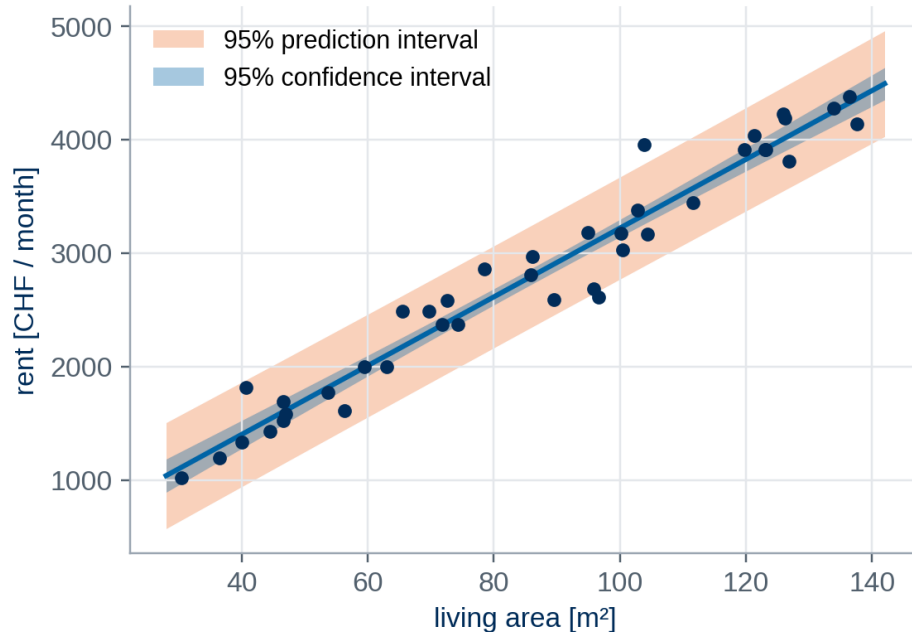
$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

in the units of y (e.g. CHF): “on average we are off by about ...”

Example: residuals 10, 5, 2, 3, 4

$$\sqrt{\frac{1}{5}(10^2 + 5^2 + 2^2 + 3^2 + 4^2)} = \sqrt{30.8} = 5.55$$

Linear regression: confidence vs prediction interval



95 % confidence interval (CI)

where the AVERAGE rent of all 95 m² flats lies

95 % prediction interval (PI)

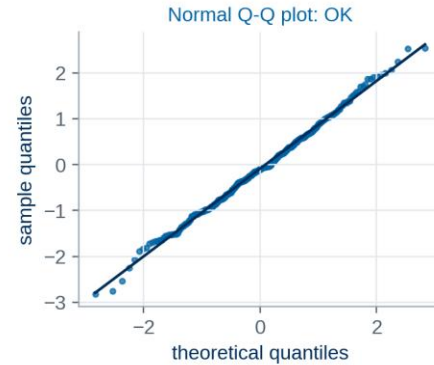
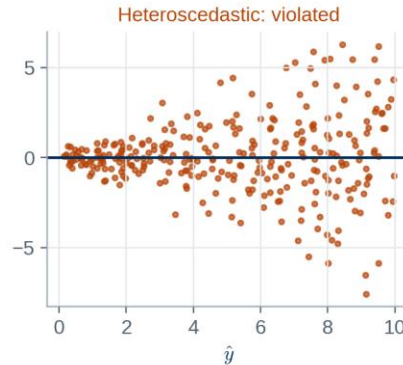
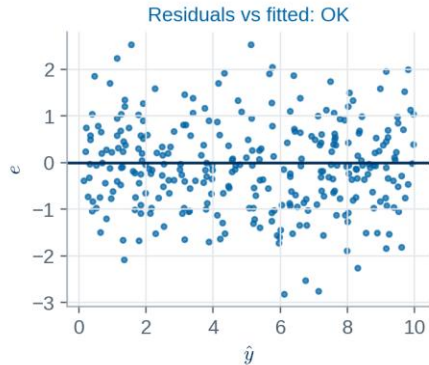
where the rent of ONE new 95 m² flat will lie

Example: 95 m² → CI ≈ 3'000–3'140 CHF, PI ≈ 2'620–3'520 CHF.

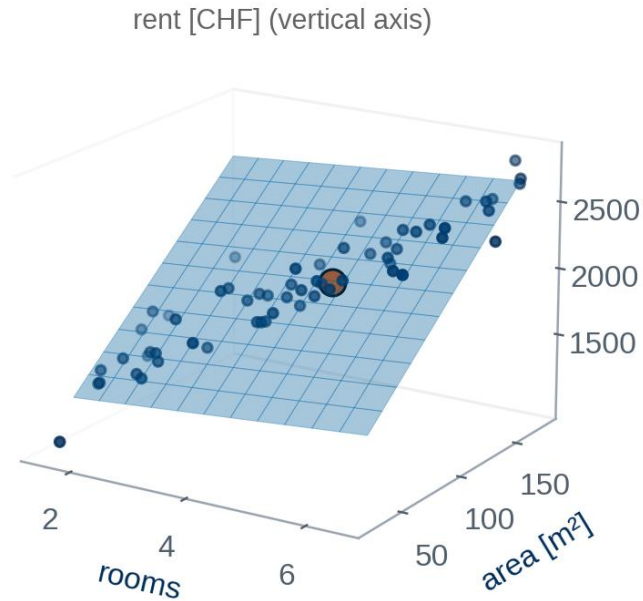
The PI is always wider: it adds the scatter of an individual flat around the line.

Linear regression: assumptions and diagnostics

Assumption	Meaning	Check
Linearity	$E[y x]$ is linear in β	residuals vs fitted
Independence	observations independent	study design, time order
Normality	$\varepsilon \sim N(0, \sigma^2)$: needed for exact small-sample t/F inference, not for OLS estimation	Q-Q plot
Equal variance	$\text{Var}(\varepsilon)$ constant (homoscedastic)	residuals vs fitted
No perfect multicollinearity	perfect: β not identifiable · strong: inflated standard errors	correlation matrix, VIF



Linear regression: multiple predictors



$$\widehat{\text{rent}} = 793.84 + 36.48 \cdot \text{rooms} + 9.80 \cdot \text{area}$$

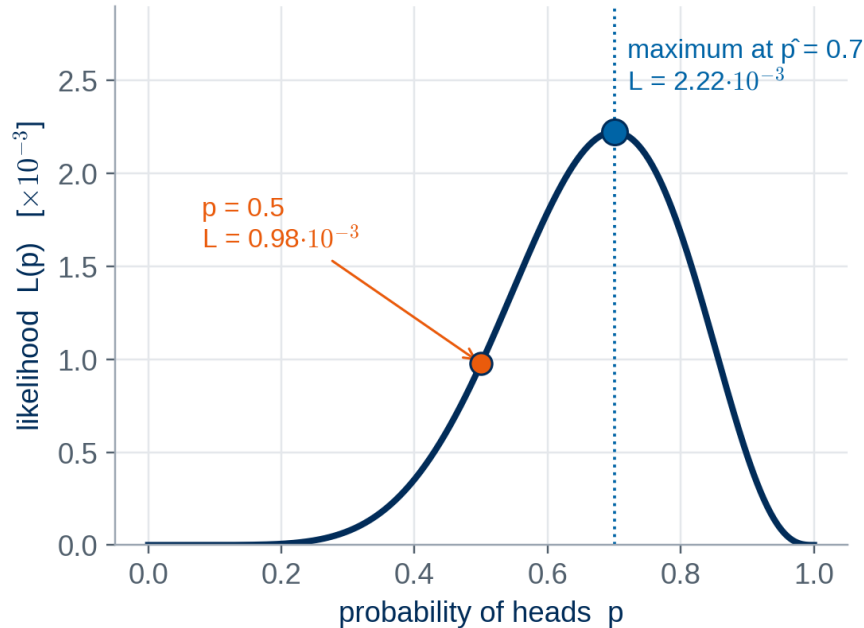
variable	coefficient	p-value
intercept	793.843	< 0.001
rooms	36.477	< 0.001
area [m²]	9.795	0.0045

$$\widehat{\text{rent}}(4.5, 110) = 793.84 + 36.48 \cdot 4.5 + 9.80 \cdot 110 \approx \mathbf{2035 \text{ CHF}}$$

Ceteris paribus: β_j = estimated change in $E[y | X]$ for +1 in x_j , holding the other included predictors constant — an association, not automatically a causal effect. Orange dot: 4.5 rooms, 110 m².

Linear regression: maximum likelihood (the idea)

Sequence with 7 heads, 3 tails: which p fits best?



Idea

Choose the parameter values under which the observed data are most plausible.

Example: a sequence of 10 tosses: 7 H, 3 T. Which p ?

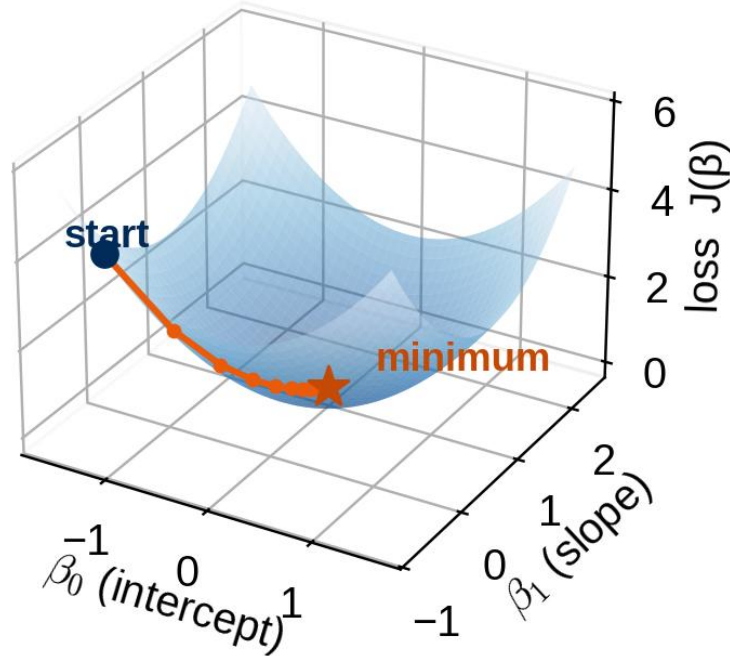
$$L(p) = p^7(1-p)^3$$

$p = 0.5 \rightarrow L = 0.00098$ $p = 0.7 \rightarrow L = 0.00222$ (max.) (for “7 heads in any order” $\times 120$ — same $\hat{p}$)

KEY IDEA For linear regression with independent normal errors of constant variance, MLE gives exactly the OLS line. Logistic regression is fitted by MLE.

Linear regression: gradient descent

Loss surface: walking downhill



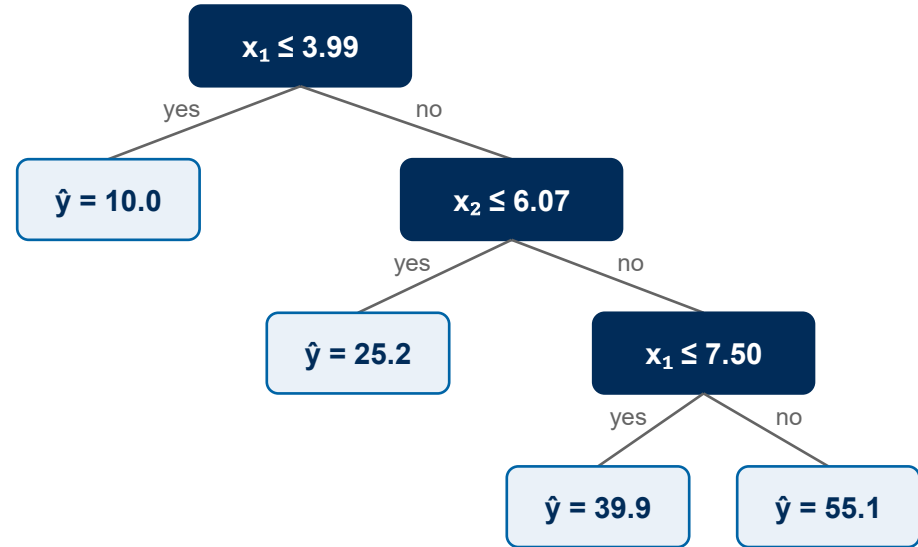
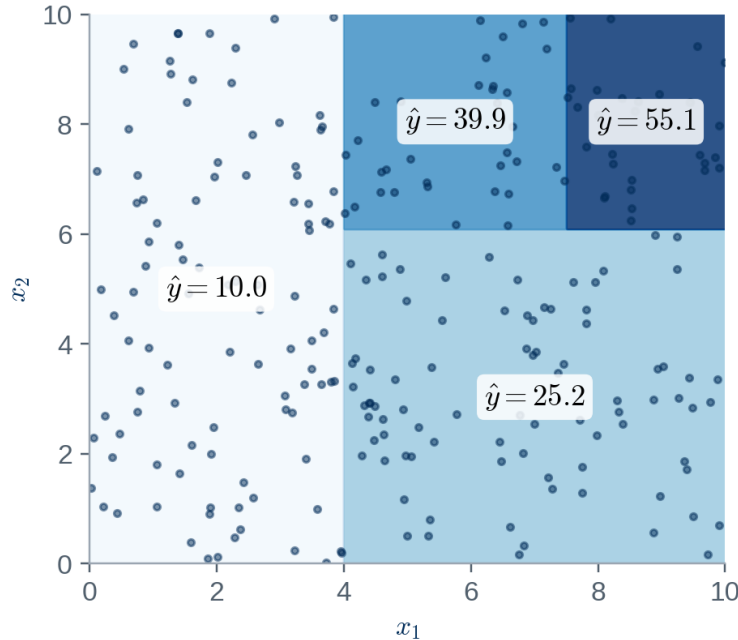
Repeat until the loss stops decreasing:

- 1 Compute the loss for the current coefficients β
- 2 Compute the slope (gradient) of the loss: in which direction does it increase?
- 3 Take a small step of size η in the opposite direction

$$\beta_{\text{new}} = \beta_{\text{old}} - \eta \cdot (\text{slope of the loss})$$

Example: like walking downhill in fog — feel the slope, step down, repeat. Steps too large (η) overshoot the valley; stochastic GD uses one observation or a mini-batch per step.

Regression trees: partitioning the feature space



In words: the tree cuts the feature space into boxes and predicts one constant per box — the mean of the training points in that box.

Regression trees: how a split is chosen

- 1 Pick a feature (e.g. area) and a threshold (e.g. 80 m²): smaller values go left, the rest right
- 2 Compute the mean of y in each child and the squared errors around it (SSE)
- 3 Try all features and thresholds; keep the split with the smallest total SSE

$$\text{SSE}(\text{left}) + \text{SSE}(\text{right}) \rightarrow \min$$

Leaf prediction = average y of the training points in the leaf

KEY IDEA Greedy and recursive: split, then repeat in each child until a stopping rule applies. Worked example on the next slide.

Regression trees: worked example of a split

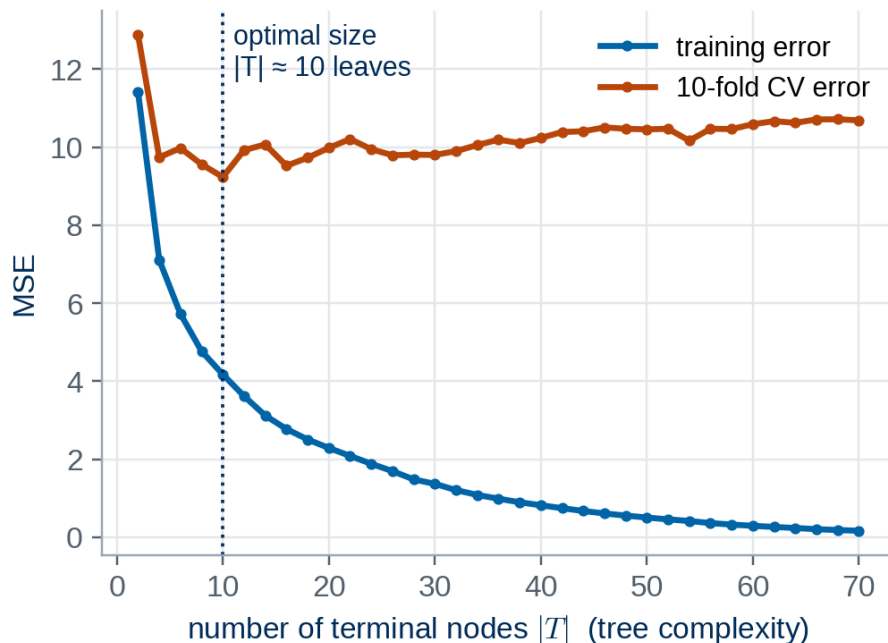
Data: $x = 1, 2, 3, 4, 5 \rightarrow y = 10, 12, 14, 30, 34$

Root node: $\bar{y} = 20$, $SSE = 100 + 64 + 36 + 100 + 196 = 496$

split at	left node R_1	mean R_1	right node R_2	mean R_2	$SSE(R_1) + SSE(R_2)$
$x \leq 1.5$	{10}	10	{12, 14, 30, 34}	22.5	$0 + 371 = 371$
$x \leq 2.5$	{10, 12}	11	{14, 30, 34}	26	$2 + 224 = 226$
$x \leq 3.5$	{10, 12, 14}	12	{30, 34}	32	$8 + 8 = 16$
$x \leq 4.5$	{10, 12, 14, 30}	16.5	{34}	34	$251 + 0 = 251$

DECISION Best split: $x \leq 3.5$ (between the 3rd and 4th observation). SSE drops from 496 to 16 (–97 %).
Leaf predictions: 12 and 32. Then repeat recursively in each child node.

Regression trees: pruning



Cost-complexity pruning

$$R_{\alpha}(T) = \underbrace{R(T)}_{\text{training error}} + \alpha \underbrace{|T|}_{\text{\#leaves}}$$

In words: error on the training data plus a fee α for every leaf. Higher fee $\rightarrow$ smaller tree.

- **$\alpha = 0$:** no penalty, largest subtree (may overfit)
- **α large:** very small tree (may underfit)
- **choose α by CV** (ccp_alpha in scikit-learn)

Breiman, Friedman, Olshen & Stone (1984)

Regression trees: hyperparameters in scikit-learn

parameter	meaning	increasing it ...
<code>max_depth</code>	maximum depth of the tree	more variance, less bias
<code>min_samples_split</code>	min. observations to split a node	less variance, more bias
<code>min_samples_leaf</code>	min. observations in a leaf	less variance, more bias
<code>max_leaf_nodes</code>	maximum number of leaves	more variance, less bias
<code>max_features</code>	features considered per split	less randomisation: stronger but more correlated trees (RF)
<code>ccp_alpha</code>	cost-complexity penalty α	smaller tree, less variance
<code>criterion</code>	squared_error · gini · entropy	—

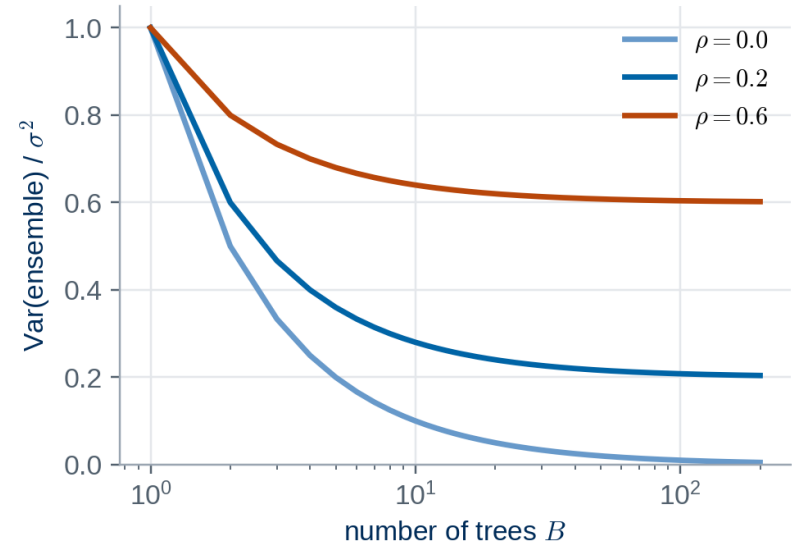
Ensembles: why averaging many models helps

variance of the average = shared part + independent part $\div B$

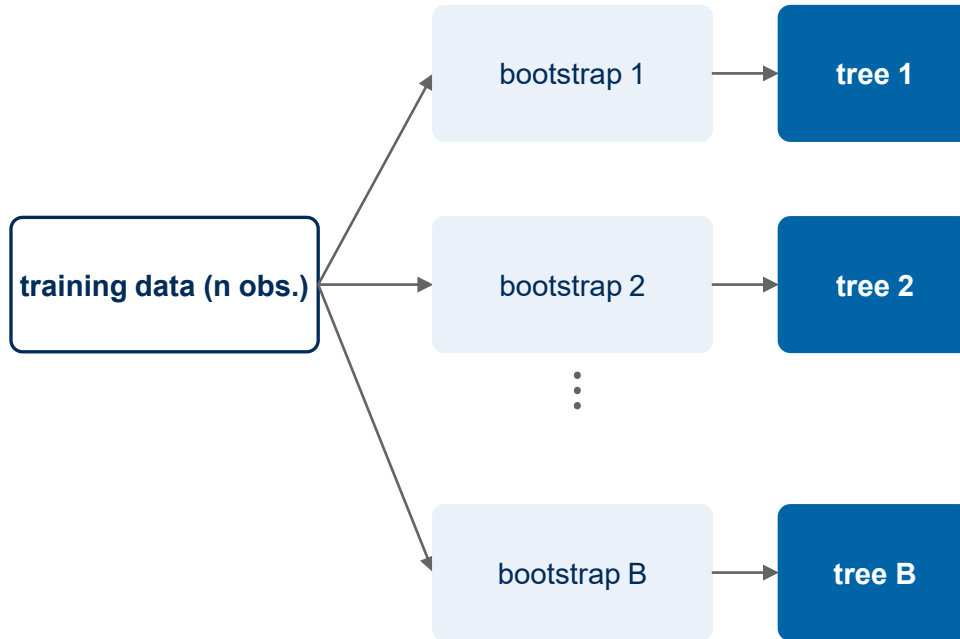
In words: assume all B models have equal variance σ^2 and common pairwise correlation ρ : averaging shrinks only the independent part; the shared part $\rho\sigma^2$ remains.

Example: $\sigma^2 = 1$, $\rho = 0.2$, $B = 100 \rightarrow 0.2 + 0.8/100 = 0.208$: averaging removes ~80 % of the variance, but never the $\rho\sigma^2$ part.

KEY IDEA Bagging mainly reduces variance; boosting reduces systematic error step by step and can affect both bias and variance.



Ensembles: bagging (bootstrap aggregation)



Regression: average the B tree predictions

$$\hat{y} = \frac{\hat{y}^{(1)} + \hat{y}^{(2)} + \dots + \hat{y}^{(B)}}{B}$$

Classification: vote or average probabilities

Example: classic: majority vote; scikit-learn: mean class probability

Out-of-bag observations

≈ 37 % of the observations are NOT drawn into a given bootstrap sample

In words: each tree never sees ~37 % of the data; these out-of-bag points give an internal error estimate without an extra validation split — not a replacement for a final test set.

Random forest (regression): bagging + random features

Random forest = bagging + random feature subsets

1

Bootstrap

draw B samples of size n with replacement

2

Random features

at every split consider only m of the p features

3

Grow deep trees

little or no pruning: low bias, high variance

4

Aggregate

average (regression); vote or mean class probability (classification)

Classic heuristics for m (Breiman; Hastie et al.)

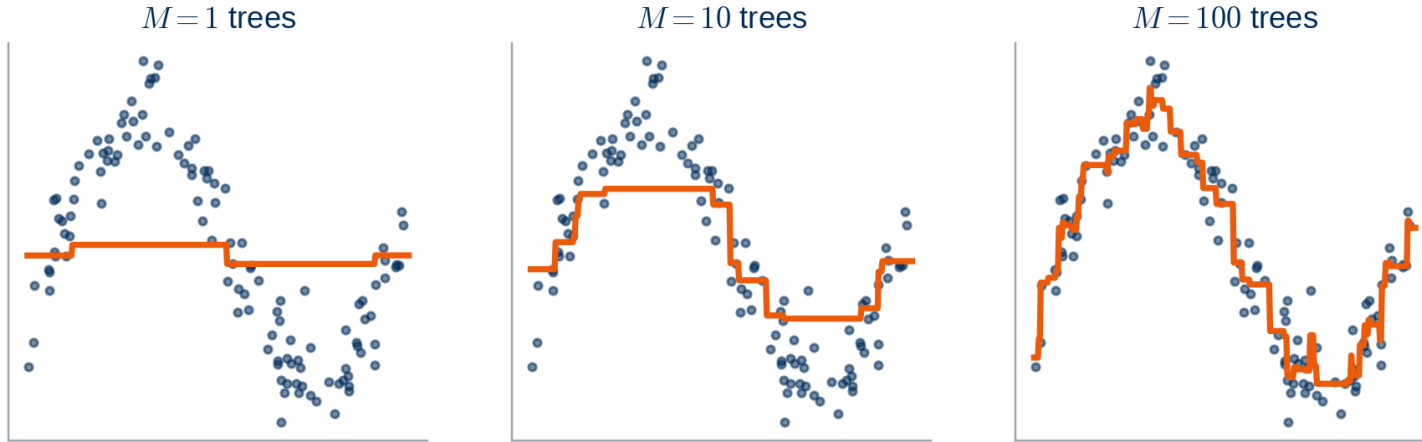
$m \approx \sqrt{p}$ (classification), $m \approx p/3$ (regression)

Example: scikit-learn defaults: classifier $\sqrt{p}$ ("sqrt"), regressor all p (1.0)

- **n_estimators:** more trees stabilise the ensemble; gains plateau while cost grows
- **feature importance:** impurity-based (MDI) can be biased → prefer permutation importance

WHY IT WORKS Random feature subsets de-correlate the trees (smaller p) → lower variance of the average.

Gradient boosting (regression): learning from errors



$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \nu h_m(\mathbf{x})$$

In words: new model = old model + a small step ν towards a new small tree h_m .

What does the next tree learn?

In words: with squared loss: the current errors (residuals) $y - \text{prediction}$. Other losses: the negative gradient of the loss (pseudo-residuals).

Example: (squared loss) $y = 10$, current prediction $7 \rightarrow$ residual 3 ; the next tree predicts ≈ 3 , with $\nu = 0.1$ the model moves to 7.3 .

Regression models compared: typical tendencies

	Linear regression	Single tree	Random forest	Gradient boosting
relationships	linear (+ chosen terms)	non-linear, interactions	non-linear, interactions	non-linear, interactions
interpretability	high (coefficients)	high (rules)	low (importance only)	low
complexity / overfitting	depends on number & choice of predictors (p vs n, interactions)	high variance if grown deep → prune	averaging substantially reduces variance	controlled by depth, learning rate v , iterations M
data needed	small	medium	larger	larger
computation	very cheap	cheap	moderate, parallel	moderate, sequential
key hyperparameters	— (or regularisation)	max_depth, ccp_alpha	n_estimators, max_features	learning_rate, n_estimators, max_depth

Typical tendencies / rules of thumb — not fixed properties. Compare candidate models with cross-validation on your data.

Classification models

Logistic regression

Classification trees

Random forest & boosting

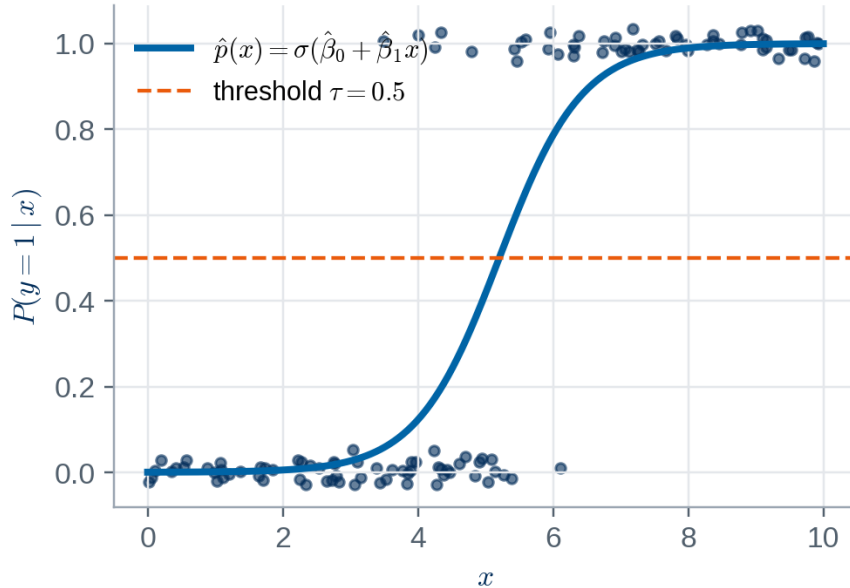
k-NN

Naive Bayes

SVM

Evaluation

Logistic regression: a linear model for classification



Model (one feature)

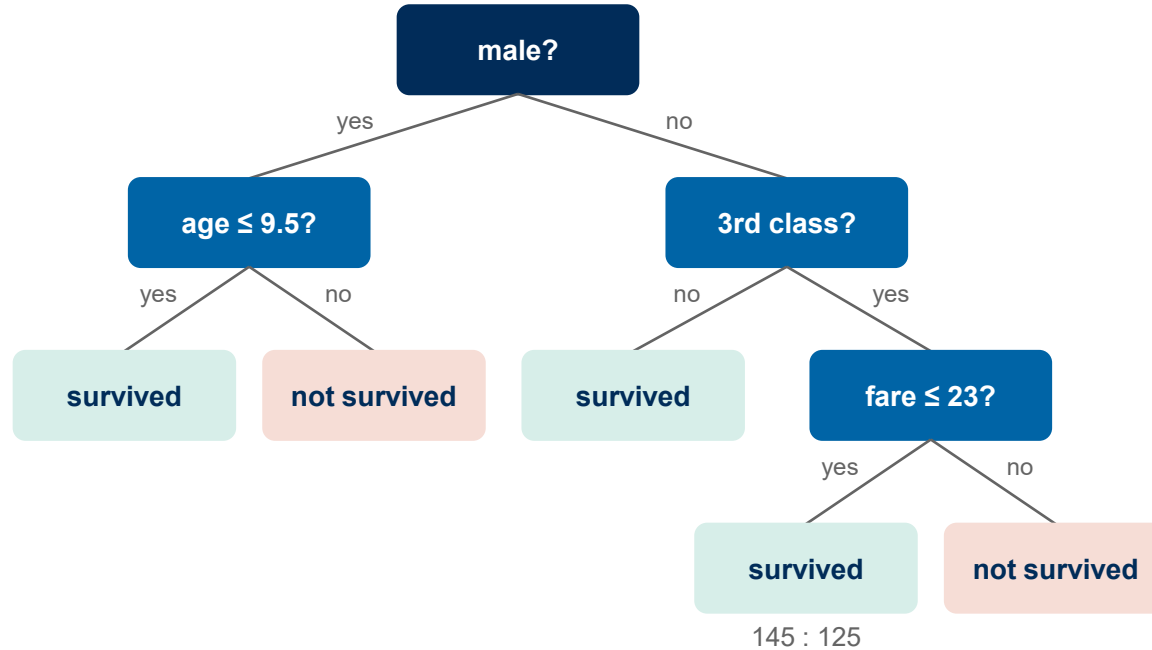
$$p(x) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x)}}$$

In words: a linear score $\beta_0 + \beta_1 x$ is squeezed by an S-curve into a probability between 0 and 1.

Example: $\beta_0 = -5$, $\beta_1 = 1$, $x = 6 \rightarrow$ score 1 $\rightarrow p = 1/(1+e^{-1}) \approx 0.73 \rightarrow$ class 1 (if $\tau = 0.5$).

- $\beta_1 > 0$: probability rises with x
- fitted by maximum likelihood (log-loss), solved numerically

Classification trees: anatomy



- root node
- internal (child) node
- terminal node (leaf)

depth d = longest root→leaf path (here 3)

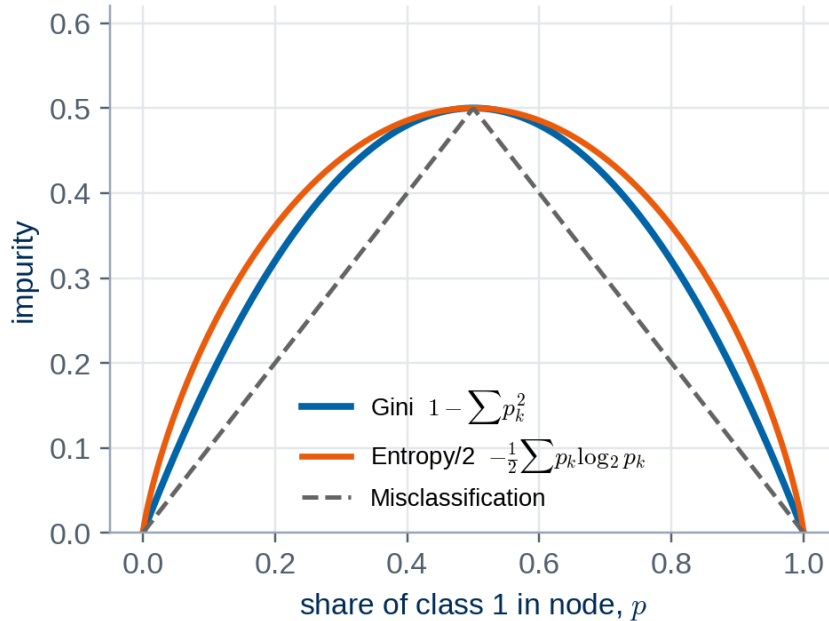
size = number of nodes (here 9)

$$\text{size} \leq 2^{d+1} - 1$$

e.g. $d = 3 \rightarrow$ at most 15 nodes

Each internal node asks a yes/no question; branches are labelled yes/no.

Classification trees: impurity (Gini)



Gini impurity

$$G = 1 - \sum_k \hat{p}_k^2$$

In words: $\hat{p}_k$ = share of class k in the node. 0 = pure node; 0.5 = maximally mixed (two classes, 50 : 50).

Example: 10 observations, yes : no

$$5:5 \Rightarrow 1 - (0.5^2 + 0.5^2) = 0.50$$

$$3:7 \Rightarrow 1 - (0.3^2 + 0.7^2) = 0.42$$

$$10:0 \Rightarrow 1 - (1^2 + 0^2) = 0.00$$

Entropy is an alternative measure with almost the same shape.

Classification trees: worked example of a split

$$\Delta G = G_{\text{parent}} - \left(\frac{n_L}{n} G_L + \frac{n_R}{n} G_R \right)$$

In words: impurity before the split minus the size-weighted impurity of the two children. Bigger ΔG = better split.

Parent node: 100 passengers, 40 survived / 60 not $\rightarrow G = 1 - (0.4^2 + 0.6^2) = 0.48$

candidate split	left child (yes)	G_L	right child (no)	G_R	weighted G	ΔG
male?	60 : 12 surv. / 48 not	0.320	40 : 28 / 12	0.420	$0.6 \cdot 0.32 + 0.4 \cdot 0.42 = 0.360$	0.120
1st class?	25 : 15 / 10	0.480	75 : 25 / 50	0.444	$0.25 \cdot 0.48 + 0.75 \cdot 0.444 = 0.453$	0.027

DECISION “male?” reduces impurity most $\rightarrow$ it becomes the root split. Repeat recursively in each child node.

Classification trees: growing and predicting

- 1 For every feature and every threshold, compute the impurity decrease of the split
- 2 Pick the (feature, threshold) with the largest decrease and split the node
- 3 Repeat recursively in each child node
- 4 Stop when a rule applies (max depth, min samples, pure node)
- 5 Optionally prune back the large tree (next slide)

Prediction in a classification leaf m

share of each class in the leaf $\rightarrow$ predict the most frequent class

Example: leaf with 145 survived, 125 not survived

$$\hat{p}(\text{survived}) = \frac{145}{145 + 125} = 0.54 \geq 0.5 \Rightarrow \text{survived}$$

KEY IDEA Trees output probabilities, not only labels — the basis for thresholds and ROC curves.

Random forest & gradient boosting (classification)

Random forest classifier

- many deep classification trees (bootstrap + random features)
- each tree gives class probabilities (shares in its leaf); the forest averages them → class with the highest mean probability

`RandomForestClassifier`

Gradient boosting classifier

- many small trees, built one after the other
- each tree fits the pseudo-residuals (negative gradient) of the log-loss

`GradientBoostingClassifier`

Example: 3 trees give $P(\text{survived}) = 0.8, 0.6, 0.3 \rightarrow \text{mean } 0.57 \rightarrow \text{predict "survived"} (\geq 0.5)$.

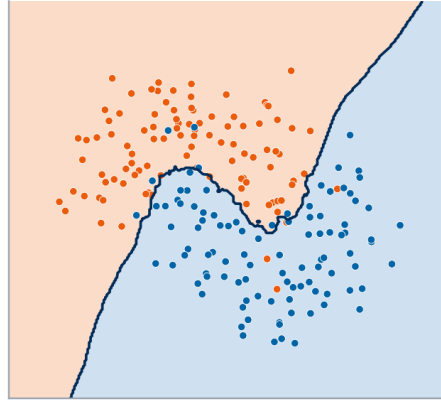
KEY IDEA Same ensemble principles as for regression (Part 3) — only the trees and the aggregation (class probabilities instead of numbers) change.

k-nearest neighbours (k-NN) classifier

$k = 1$ (high variance)



$k = 15$ (smoother)



- Choose k
- Compute distances to all training points
- Take the k nearest: $\mathcal{N}_k(x)$
- Predict the majority class (or mean of y)

Distance

$$d(\mathbf{x}, \mathbf{x}') = \sqrt{\sum_{j=1}^p (x_j - x'_j)^2}$$

In words: straight-line distance, as with a ruler.

Class probability

share of the k nearest neighbours that belong to class c

Example: $k = 5$, three neighbours blue $\rightarrow P(\text{blue}) = 3/5 = 0.6$.

CAUTION Scale features when their units are not comparable — otherwise the largest-scale variable dominates the distance.

Naive Bayes classifier

Bayes' theorem (Thomas Bayes, 1701–1761)

$$P(y \mid \mathbf{x}) = \frac{P(\mathbf{x} \mid y) P(y)}{P(\mathbf{x})}$$

In words: how likely is class y after seeing $\mathbf{x}$? Start from the base rate $P(y)$ and update it by how typical $\mathbf{x}$ is for y .

“Naive”: features independent given the class

$$\hat{y} = \arg \max_y P(y) \prod_{j=1}^p P(x_j \mid y)$$

In words: multiply the base rate with one probability per feature; predict the class with the highest score.

Example: play golf?

new day: outlook = sunny, temp = cool, humidity = high, windy = true

$$\text{score}(\text{yes}) = \frac{9}{14} \cdot \frac{2}{9} \cdot \frac{3}{9} \cdot \frac{3}{9} \cdot \frac{3}{9} = 0.0053$$

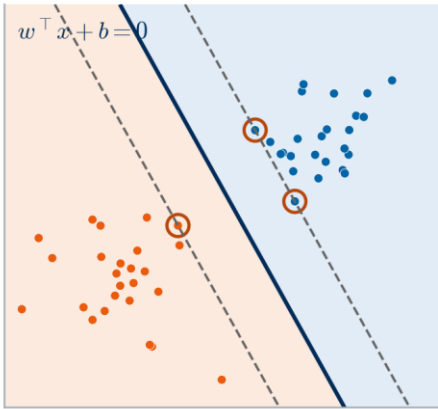
$$\text{score}(\text{no}) = \frac{5}{14} \cdot \frac{3}{5} \cdot \frac{1}{5} \cdot \frac{4}{5} \cdot \frac{3}{5} = 0.0206$$

9/14 = share of ‘yes’ days; 2/9 = share of sunny days among the ‘yes’ days, ...

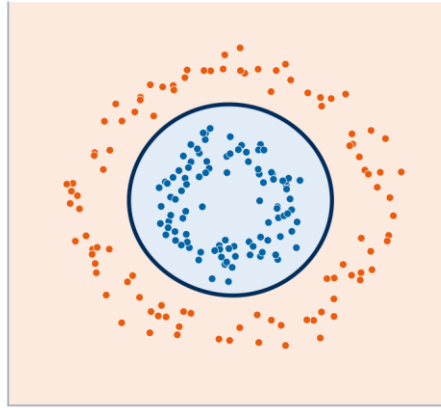
RESULT $\text{score}(\text{no}) = 0.0206 > \text{score}(\text{yes}) = 0.0053 \rightarrow \text{predict “no”}$. Normalised: $P(\text{no} \mid \mathbf{x}) = 0.0206 / (0.0206 + 0.0053) \approx 0.795$

Support vector machines (SVM) classifier

Linear SVM: maximal margin



RBF kernel: non-linear boundary



Interactive demo: jgreitemann.github.io/svm-demo

- with a soft margin, support vectors may lie inside the gap or be misclassified
- kernel trick → non-linear boundaries

Widest gap

find the line that separates the classes with the largest margin

Support vectors

the points closest to the line (circled) determine it — all others do not matter

C (soft margin)

small C: wide gap, some errors allowed · large C: few errors, narrow gap

Kernel

measures similarity in a transformed space → curved boundaries (right plot)

Evaluation: confusion matrix and metrics

		predicted	
		negative	positive
actual	neg.	TN 75	FP 22
	pos.	FN 24	TP 50

n = 171 test observations

CAUTION With imbalanced classes, accuracy alone misleads.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} = \frac{125}{171} = 0.73$$

share of all predictions that are correct

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{50}{72} = 0.69$$

of all predicted positives, how many are really positive?

$$\text{Recall (TPR)} = \frac{TP}{TP + FN} = \frac{50}{74} = 0.68$$

of all real positives, how many did we find?

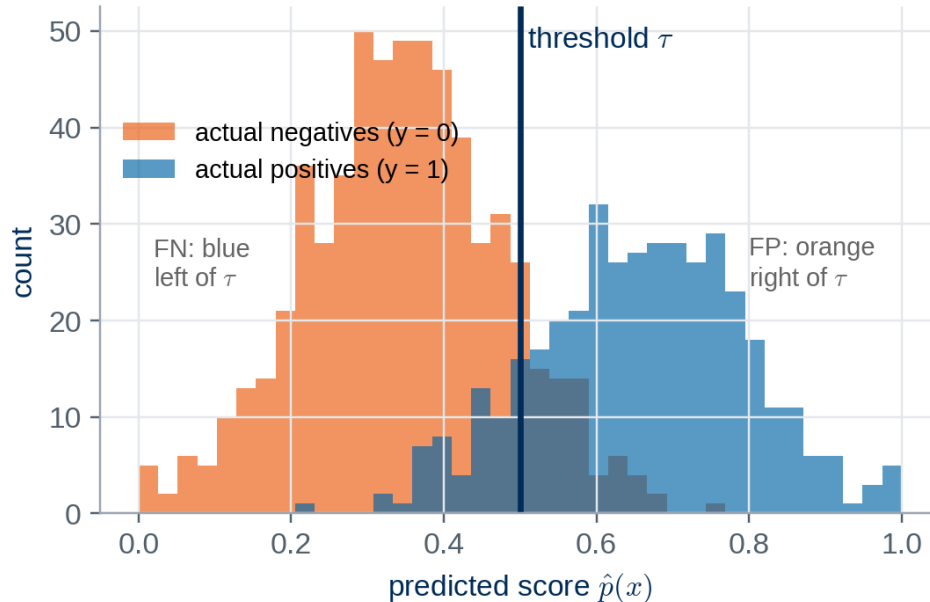
$$\text{FPR} = \frac{FP}{FP + TN} = \frac{22}{97} = 0.23$$

of all real negatives, how many were wrongly flagged?

$$F_1 = \frac{2 \cdot \text{Prec} \cdot \text{Rec}}{\text{Prec} + \text{Rec}} = 0.68$$

one number balancing precision and recall

Evaluation: classification thresholds



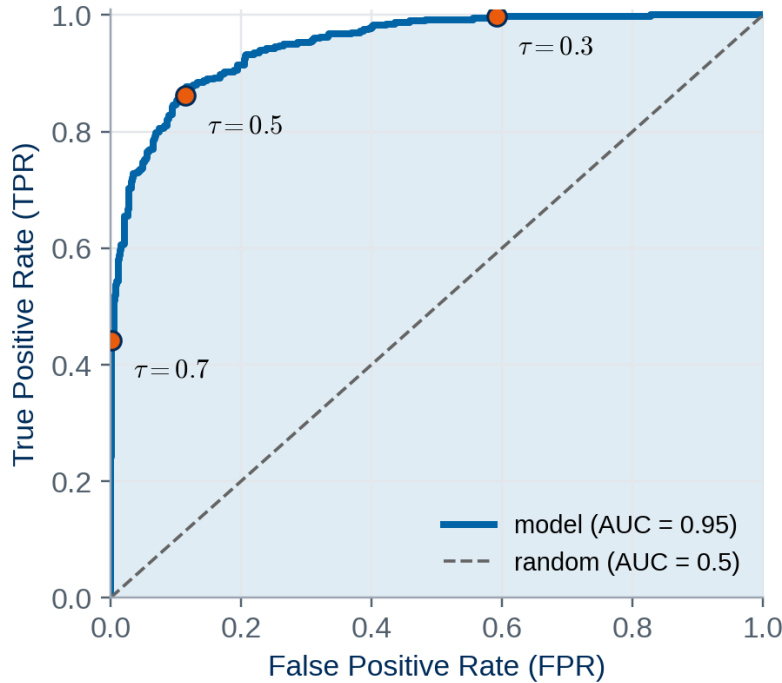
$$\hat{y} = \mathbf{1}\{\hat{p}(\mathbf{x}) \geq \tau\}$$

In words: predict “positive” when the predicted probability is at least τ .

- $\tau \uparrow$ fewer predicted positives: recall and FPR cannot increase; precision often rises, but not guaranteed
- $\tau \downarrow$ more predicted positives: recall and FPR cannot decrease
- **default** $\tau = 0.5$ is common for probability outputs; score-based classifiers use an analogous threshold

RULE Pick τ from the costs of false positives vs false negatives.

Evaluation: ROC curve and AUC



ROC curve

For every threshold τ : plot (FPR, TPR) — then connect the points.

one point per threshold τ ; diagonal = random ranking (in expectation)

Area under the curve

AUC = area under the ROC curve (0.5 = random ranking, 1 = perfect)

In words: the probability that a random positive case gets a higher score than a random negative case (ties count $\frac{1}{2}$).

Example: AUC = 0.95 $\rightarrow$ in 95 % of such pairs the positive case is ranked higher (0.5 = coin flip, 1 = perfect).

Unsupervised learning

Overview

k-means

Hierarchical clustering

PCA

Unsupervised learning: no target variable

	Supervised (Parts 3–4)	Unsupervised (Part 5)
data	features x AND known answer y	only features x
goal	predict y for new cases	find groups or a compact description
evaluation	test error (RMSE, accuracy, AUC)	no target used for fitting: internal criteria, stability, interpretation (external labels sometimes available)
typical use	rent prediction, churn yes/no	customer segments, anomaly detection, visualisation

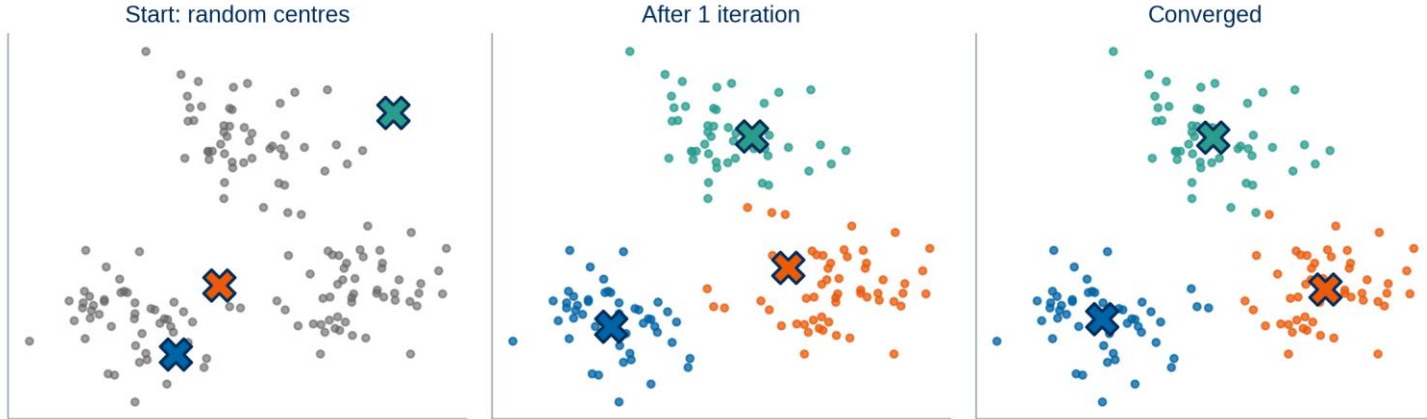
Clustering

group similar observations: k-means, hierarchical clustering

Dimension reduction

summarise many features by a few new ones: PCA

k-means clustering: the algorithm

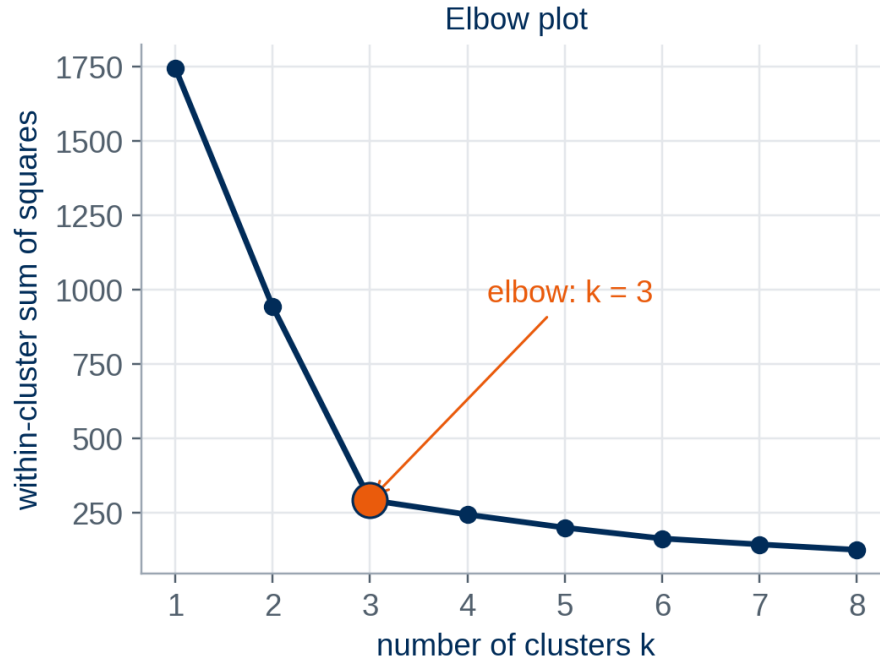


- 1 Choose the number of clusters k
- 3 Assign every point to its nearest centre

- 2 Place k centres at random
- 4 Move each centre to the mean of its points — repeat 3–4 until (almost) nothing changes

In words: k-means makes the distances of points to their own cluster centre as small as possible. In Python: `KMeans(n_clusters=3, n_init=10)`

k-means clustering: choosing k (elbow method)

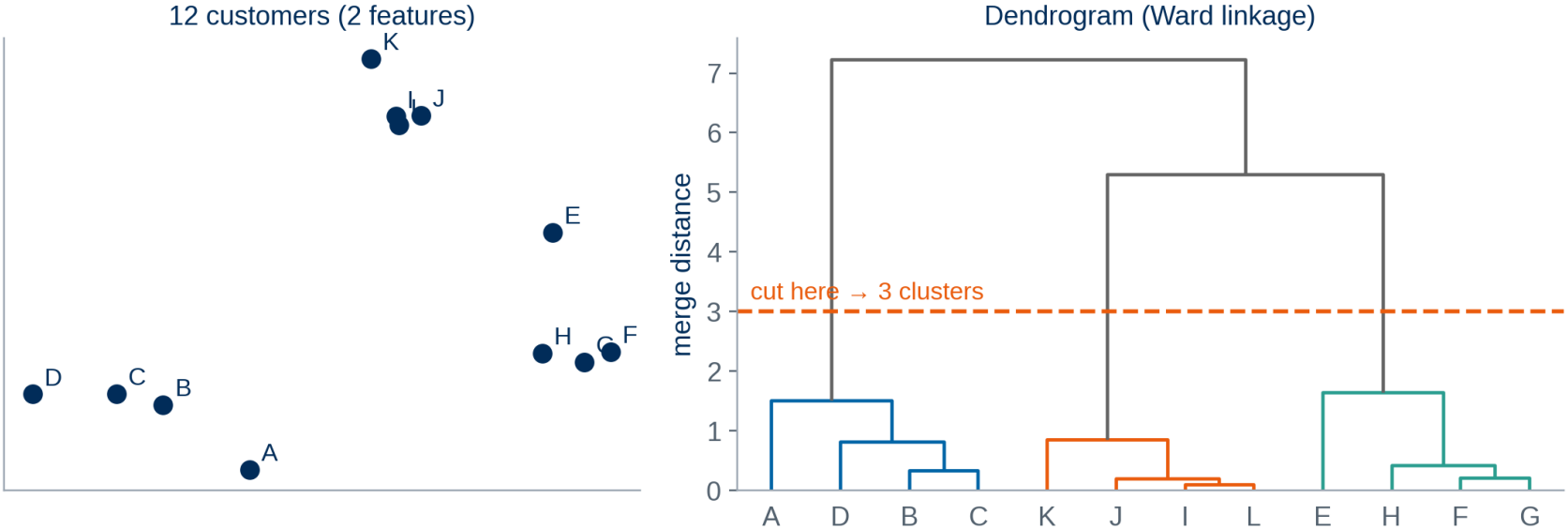


Reading the elbow plot

- y-axis: how spread out the points are within their clusters (smaller = tighter clusters)
- more clusters always reduce it — $k = n$ would give 0
- choose k where the curve bends: here $k = 3$

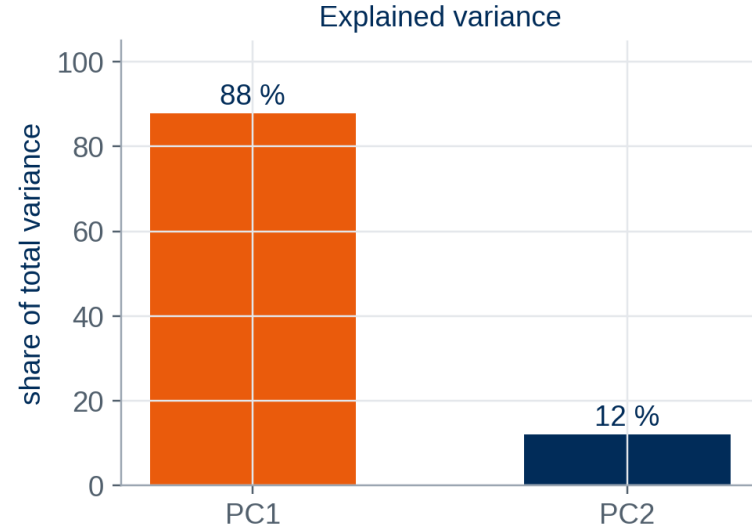
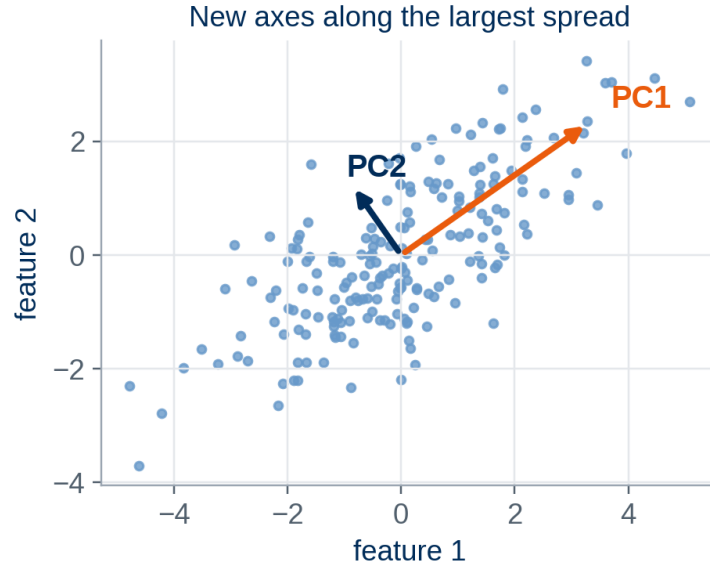
CAUTION Scale the features first, use several random starts, and check that the clusters make sense for the business question.

Hierarchical clustering: the dendrogram



- 1 Start: every customer is its own cluster
- 2 Merge the two closest clusters; repeat until one cluster is left
- 3 Cut the dendrogram at a height $\rightarrow$ clusters (here 3)

PCA – principal component analysis (outlook)



In words: PCA rotates the coordinate system so that the first new axis (PC1) captures as much of the spread as possible. Here PC1 explains 88 % of the total variance of the two features — they can be summarised well by one.

Putting it together in scikit-learn

```
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import confusion_matrix, roc_auc_score

X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)

grid = GridSearchCV(RandomForestClassifier(random_state=42),
    {"max_features": ["sqrt", 0.5],
     "min_samples_leaf": [1, 5, 20]},
    cv=5, scoring="roc_auc")
grid.fit(X_tr, y_tr)

p = grid.predict_proba(X_te)[:, 1]
print(roc_auc_score(y_te, p))
print(confusion_matrix(y_te, p >= 0.5))
```

`train_test_split`

Part 2 · hold-out

`GridSearchCV(cv=5)`

Part 2 · k-fold CV

`RandomForestClassifier`

Part 4 · classification ensembles

`min_samples_leaf`

Part 3 · tree complexity

`predict_proba, ≥ 0.5`

Part 4 · thresholds

`roc_auc_score`

Part 4 · ROC / AUC

Learning objectives

You can ...

- explain ML, its paradigms and the difference between regression, classification and unsupervised learning
- explain train/test split, cross-validation, over-/underfitting and the bias–variance trade-off
- explain how OLS fits a line and interpret coefficients of simple and multiple regression
- compute and interpret R^2 , RMSE, confidence and prediction intervals; check assumptions
- explain the ideas of maximum likelihood and gradient descent
- explain logistic regression as a probabilistic linear classifier
- construct a tree by hand using variance reduction or Gini; explain pruning
- explain bagging, random forests and boosting and why ensembles reduce error
- explain k-NN, Naive Bayes and SVM and compute a Naive Bayes example
- compute confusion-matrix metrics; interpret thresholds, ROC curves and AUC
- apply k-means, read an elbow plot and a dendrogram, and explain the idea of PCA

Literature

- Braschler, M., Stadelmann, T. & Stockinger, K. (2019). Applied Data Science. Springer.
- Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5–32.
- Breiman, L., Friedman, J., Olshen, R. & Stone, C. (1984). Classification and Regression Trees. Wadsworth.
- Bruce, P., Bruce, A. & Gedeck, P. (2020). Practical Statistics for Data Scientists (2nd ed.). O'Reilly.
- Hastie, T., Tibshirani, R. & Friedman, J. (2009). The Elements of Statistical Learning (2nd ed.). Springer.
- James, G., Witten, D., Hastie, T. & Tibshirani, R. (2021). An Introduction to Statistical Learning (2nd ed.). Springer.
- Li, H. (2024). Machine Learning Methods. Springer.
- Mitchell, T. (1997). Machine Learning. McGraw-Hill.